

Resumo Executivo do Passo 1.4: Estabelecimento da camada de governança tipográfica, normalização de controle de versão e blindagem de segredos corporativos. O repositório passa a contar com o **.editorconfig** para sincronização universal entre IDEs, o **.gitattributes** implementando a **ADR 007** (blindagem de quebras de linha LF para containers Linux) e o contrato declarativo **.env.example** cobrindo todas as 7 camadas da arquitetura SCSi. Todos os artefatos foram auditados no nível de bytes (100% UTF-8 sem BOM e terminação LF).

1. Matriz de Entregas por Micro-Fase

Micro-Fase	Escopo & Atribuição	Detalhamento da Implementação Técnica	Avaliação
1.4.1	Padronização de Editores	Criação do .editorconfig com root = true, charset UTF-8, terminação LF, 4 espaços para Python (88 colunas) e 2 espaços para configurações estruturadas (YAML, JSON, TOML, Docker).	✓ Aprovado
1.4.2	Blindagem Git (ADR 007)	Criação do .gitattributes forçando *.sh text eol=lf e Dockerfile* text eol=lf. Neutraliza o core.autocrlf=true do Windows e impede falhas de shebang nos containers Linux da VPS.	✓ Aprovado
1.4.3	Contrato de Variáveis	Criação do .env.example corporativo com documentação de 7 camadas da stack: Django Core, PostgreSQL 16, Redis Cache, RabbitMQ AMQP, Celery Worker/Beat, IA (OpenAI, Gemini, Anthropic, LangSmith) e Traefik/SSL.	✓ Aprovado
1.4.4	Auditoria de Conformidade	Inspeção binária confirmando 6/6 arquivos em UTF-8 limpo (sem BOM) e LF puro. Prova matemática de bloqueio de segredos no .gitignore (bloqueia .env*, rastreia .env.example). Pipeline ruff/pytest 100% verde.	✓ Aprovado
1.4.5	Emissão do Relatório PDF	Compilação e validação do relatório executivo em alta resolução, sincronização em docs/ e salvaguarda definitiva na memória viva do projeto.	✓ Aprovado

2. Os Três Pilares de Guardrails do Repositório

 **.editorconfig**

- **Universal:** Respeitado por VS Code, Cursor, PyCharm e Neovim.
- **Alinhamento:** Espaçamento de 4 colunas em Python sincronizado com o Ruff.
- **Exceção Markdown:** Preserva espaços finais para quebras de linha (
).

 **.gitattributes (ADR 007)**

- **Anti-CRLF:** Força terminação LF em scripts bash mesmo no Windows.
- **Proteção de Shebang:** Impede erro /bin/sh^M: bad interpreter.
- **Binários Imutáveis:** Bloqueia corrupção de PDFs, imagens e fontes.

 **.env.example**

- **Contrato SCSi:** Mapeia todas as portas, vhosts e credenciais da stack.
- **Dual-Pattern DB:** Suporta injeção direta no Compose e URL unificada.
- **Multi-LLM & Tracing:** OpenAI, Gemini, Claude e LangSmith integrados.

3. Matriz de Auditoria Física de Bytes e Encodings

Arquivo de Governança	Tamanho	Assinatura de Encoding	Terminação de Linha	Avaliação
.editorconfig	1.321 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme
.gitattributes	1.617 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme
.env.example	4.004 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme
pyproject.toml	3.444 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme
.gitignore	916 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme
.python-version	5 bytes	UTF-8 (Clean / Zero BOM)	LF (Unix 0x0A puro)	✓ 100% Conforme

4. Teste de Blindagem Lógica de Segredos (.gitignore)

```
PS C:\estudo_arquitetura_sistemas_pycoder> uv run python -c "verificar_padroes_gitignore()" .env -> IGNORED  
(Blocked) [Credencial real blindada] .env.local -> IGNORED (Blocked) [Credencial de desenvolvimento blindada]  
.env.production -> IGNORED (Blocked) [Credencial de produção blindada] .env.example -> TRACKED (Allowed) [Contrato  
modelo de referência liberado] settings.py -> TRACKED (Allowed) [Código-fonte da aplicação liberado]
```